



Database systems

A database systems consists of set of shared data objects that can be accessed by users. A data object can be a page, a file, a segment or a record.

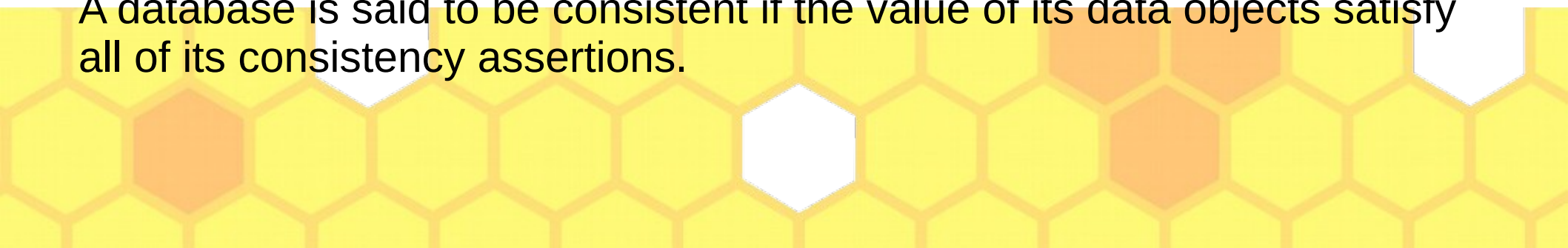
For the purpose of concurrency control , we will view database as a collection of data objects. ($d_1, d_2, d_3, \dots, d_M$).

Each data objects takes values from a specified domain.

The state of a database is given by the value of its data objects.

In a database certain semantic relationship, called consistency assertions or integrity constraints must hold among its data objects.

A database is said to be consistent if the value of its data objects satisfy all of its consistency assertions.



Transactions

- A user interacts with a database by performing read and write actions on the data objects. The actions of a user are normally grouped together to form a single unit of interaction , termed as transaction.
- A transaction consists of a sequence of read, compute and write statements that refer to the data objects of a database.
- A transaction that does not modify any data objects but just reads some of them is called read only transaction or a query.
- A transaction that modifies at least one data object is known as update transaction.
- For a transaction , the set of data objects that are read by it are referred as its readset and the set of data objects that are written by it are referred to as its writeset.

Conflicts

- Transactions conflicts if they access the same data objects.

Transaction processing

- A transaction is executed by executing its actions one by one from the beginning to the end.
- A read action of a transaction is executed by reading the data object in the workspace of the transaction. (area, or pages)
- A write action of a transaction modifies a data object in the workspace and eventually writes it to the database.
- A transaction can be considered as function whose inputs are the values of the data objects in its readset and the outputs are the values of the data objects in its writeset.

The problem of concurrency control

- For the purpose of concurrency control we can view a database system as consisting of three software modules: A transaction manager(TM), a data manager(DM), and a scheduler.
- A TM supervises the execution of a transaction. It intercepts and executes all submitted transactions. A TM interacts with DM to carry out the execution of a transaction

- A DM manages the database. It carries out the read and write requests issued by the TM on behalf of a transaction by operating them on the database.
- The scheduler is responsible for enforcing concurrency control. It grants or releases locks on data objects as requested by a transaction.
- Typically , in a database system, several transactions are under execution simultaneously. Since a transaction preserves database consistency , a database system can guarantee consistency by executing transactions serially that is one at a time. However such a serial execution of transactions is inefficient.
- Efficiency can be improved by executing transactions concurrently. That is executing read and write actions from several transactions in an interleaved manner.
- 1. Inconsistent retrieval

Inconsistent retrieval occurs when a transaction reads some data objects of a database before another transaction has completed with

Its modification of those data objects. In such situations , the former transaction faces the risk of retrieving incorrect values of the data objects.

- 2. Inconsistent update

Inconsistent update occurs when many transactions read and write on to common set of data objects of a database, leaving the database in an inconsistent state. Thus if the interleaving of the actions of transactions is not controlled, some transactions may see an inconsistent state of the database and the database may be left in an inconsistent state. This fundamental problem is referred to as the concurrency control problem.

SERIALIZABILITY

When multiple transactions are being executed by the operating system in a multiprogramming environment, there are possibilities that instructions of one transactions are interleaved with some other transaction.



Schedule – A chronological execution sequence of a transaction is called a schedule. A schedule can have many transactions in it, each comprising of a number of instructions/tasks.

Serial Schedule – It is a schedule in which transactions are aligned in such a way that one transaction is executed first. When the first transaction completes its cycle, then the next transaction is executed. Transactions are ordered one after the other. This type of schedule is called a serial schedule, as transactions are executed in a serial manner.

In a multi-transaction environment, serial schedules are considered as a benchmark. The execution sequence of an instruction in a transaction cannot be changed, but two transactions can have their instructions executed in a random fashion. This execution does no harm if two transactions are mutually independent and working on different segments of data; but in case these two transactions are working on the same data, then the results may vary. This ever-varying result may bring the database to an inconsistent state.

To resolve this problem, we allow parallel execution of a transaction schedule, if its transactions are either serializable or have some equivalence relation among them.

Basic Synchronization primitives for concurrency control

- A concurrency control algorithm controls the interleaving of conflicting actions of transactions so that the integrity of database is maintained.

1. Locks

In lock based techniques, each data object has a lock associated with it. A transaction can request, hold, or release the lock on a data object.

- When a transaction holds a lock, the transaction is said to have locked the corresponding data object. A transaction can lock a data object in two modes:- exclusive and shared.
- If a transaction has locked a data object in exclusive mode, no other transaction can concurrently lock it in any mode.
- If a transaction has locked a data object in shared mode, other transaction can concurrently lock it but only in shared mode.
- By locking data objects , a transaction ensures that the locked data objects are inaccessible to other transactions, while temporarily in inconsistent states.

2. Timestamps

A Timestamp is a unique number that is assigned to a transaction or a data object and is chosen from a monotonically increasing sequence. Time stamps are commonly generated according to Lamport's scheme.

- Every Site S_i has a logical clock C_i which takes monotonically non decreasing integer values. When a transaction T is submitted at a site S_i , S_i increments C_i by one and then assign a 2-tuple (C_i, i) to T . The 2-tuple is referred to as the timestamp of T and is denoted by $TS(T)$.
- Timestamp has two properties:- 1. Uniqueness- Timestamp generated by different sites differ in their site id part and timestamps generated by the same site differ in their clock value part. 2. monotonicity:- A site generates timestamps in increasing order.
- Timestamps allows us to place a total ordering on the transactions of a distributed system to simply ordering the transactions by their timestamps.



Lock based algorithms

Concurrency Control in Database Management System is a procedure of managing simultaneous operations without conflicting with each other. It ensures that Database transactions are performed concurrently and accurately to produce correct results without violating data integrity of the respective Database.

Lock Based algorithm is a mechanism in which a transaction cannot Read or Write the data until it acquires an appropriate lock. Lock based protocols help to eliminate the concurrency problem in DBMS for simultaneous transactions by locking or isolating a particular transaction to a single user.

A lock is a data variable which is associated with a data item. This lock signifies that operations that can be performed on the data item. Locks in DBMS help synchronize access to the database items by concurrent transactions.

All lock requests are made to the concurrency-control manager. Transactions proceed only once the lock request is granted.

A transaction is well formed if it

- Locks a data object before accessing it
- Does not lock a data object more than once
- Unlocks all the locked data objects before it completes.

Binary Locks: A Binary lock on a data item can either be locked or unlocked.

Shared Lock (S): A shared lock is also called a Read-only lock. With the shared lock, the data item can be shared between transactions. This is because you will never have permission to update data on the data item.

Exclusive Lock (X): With the Exclusive Lock, a data item can be read as well as written. This is exclusive and can't be held concurrently on the same data item. X-lock is requested using lock-x instruction.

Transactions may unlock the data item after finishing the 'write' operation.

Simplistic Lock :- This type of lock-based protocols allows transactions to obtain a lock on every object before beginning operation.

Transactions may unlock the data item after finishing the 'write' operation.

Pre-claiming Locking :- Pre-claiming lock protocol helps to evaluate operations and create a list of required data items which are needed to initiate an execution process. In the situation when all locks are granted, the transaction executes. After that, all locks release when all of its operations are over.

Two Phase Locking:- Two Phase Locking Protocol also known as 2PL protocol is a method of concurrency control in DBMS that ensures serializability by applying a lock to the transaction data which blocks other transactions to access the same data simultaneously.

This locking protocol divides the execution phase of a transaction into three different parts.

- In the first phase, when the transaction begins to execute, it requires permission for the locks it needs.
- The second part is where the transaction obtains all the locks. When a transaction releases its first lock, the third phase starts.
- In this third phase, the transaction cannot demand any new locks. Instead, it only releases the acquired locks.

The Two-Phase Locking protocol allows each transaction to make a lock or unlock request in two steps:

- Growing Phase: In this phase transaction may obtain locks but may not release any locks.
- Shrinking Phase: In this phase, a transaction may release locks but not obtain any new lock

Time Stamp based Algorithm

Timestamp based algorithm which uses the System Time or Logical Counter as a timestamp to serialize the execution of concurrent transactions. The Timestamp-based protocol ensures that every conflicting read and write operations are executed in a timestamp order.

The older transaction is always given priority in this method. It uses system time to determine the time stamp of the transaction. This is the most commonly used concurrency protocol.

Lock-based protocols help you to manage the order between the conflicting transactions when they will execute. Timestamp-based protocols manage conflicts as soon as an operation is created.

Timestamp is a unique identifier created by the DBMS to identify a transaction. They are usually assigned in the order in which they are submitted to the system

Timestamp Ordering Protocol –

The main idea for this protocol is to order the transactions based on their Timestamps. A schedule in which the transactions participate is then serializable and the only equivalent serial schedule permitted has the transactions in the order of their Timestamp Values. Stating simply, the schedule is equivalent to the particular Serial Order corresponding to the order of the Transaction timestamps. Algorithm must ensure that, for each items accessed by Conflicting Operations in the schedule, the order in which the item is accessed does not violate the ordering. To ensure this, use two Timestamp Values relating to each database item X.

- $W_TS(X)$ is the largest timestamp of any transaction that executed $write(X)$ successfully.
- $R_TS(X)$ is the largest timestamp of any transaction that executed $read(X)$ successfully.

Basic Timestamp ordering

Every transaction is issued a timestamp based on when it enters the system. Suppose, if an old transaction T_i has timestamp $TS(T_i)$, a new transaction T_j is assigned timestamp $TS(T_j)$ such that $TS(T_i) < TS(T_j)$. The protocol manages concurrent execution such that the timestamps determine the serializability order.

- The timestamp ordering protocol ensures that any conflicting read and write operations are executed in timestamp order. Whenever some Transaction T tries to issue a $R_item(X)$ or a $W_item(X)$, the Basic TO algorithm compares the timestamp of T with $R_TS(X)$ & $W_TS(X)$ to ensure that the Timestamp order is not violated. This describe the Basic TO protocol in following two cases.
 1. Whenever a Transaction T issues a $W_item(X)$ operation, check the following conditions:

If $R_TS(X) > TS(T)$ or if $W_TS(X) > TS(T)$, then abort and rollback T and reject the operation. else,
Execute $W_item(X)$ operation of T and set $W_TS(X)$ to $TS(T)$.

2. Whenever a Transaction T issues a $R_item(X)$ operation, check the following conditions:

If $W_TS(X) > TS(T)$, then abort and reject T and reject the operation, else

If $W_TS(X) \leq TS(T)$, then execute the $R_item(X)$ operation of T and set $R_TS(X)$ to the larger of $TS(T)$ and current $R_TS(X)$.

Strict Timestamp Ordering –

A variation of Basic TO is called Strict TO ensures that the schedules are both Strict and Conflict Serializable. In this variation, a Transaction T that issues a $R_item(X)$ or $W_item(X)$ such that $TS(T) > W_TS(X)$ has its read or write operation delayed until the Transaction T' that wrote the values of X has committed or aborted.



Optimistic algorithm

Optimistic Concurrency Control Algorithms are based on the assumption that conflicts do not occur during execution time. No synchronization is performed when a transaction is executed.

However, a check is performed at the end of the transaction to make sure that no conflicts have occurred. If there is a conflict, the transaction will be aborted. Otherwise, the transaction is committed. Since conflicts do not occur very often, this algorithm is very efficient compared to other locking algorithms.

Kung-Robinson Algorithm

Kung and Robinson were the first to propose an optimistic method for concurrency control.

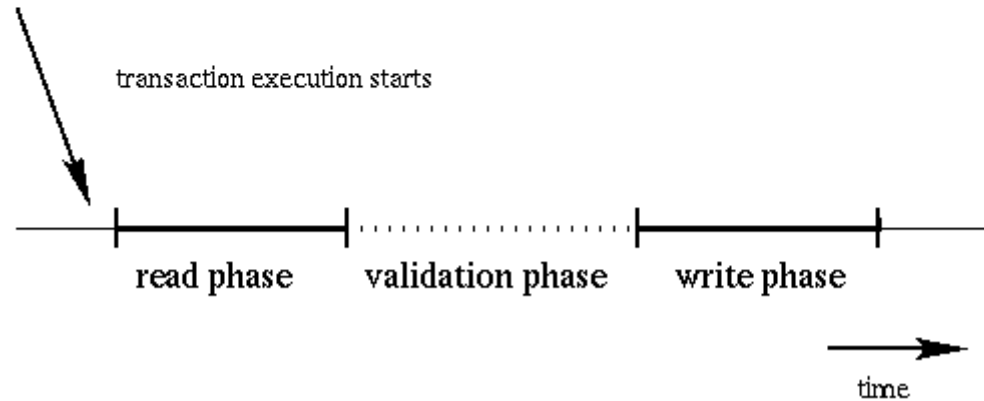
The optimistic situation for this algorithm happens when

- conflicts are unlikely to happen
- the system consists mainly read-only transactions (such as a query dominant system)

Basic Idea: No synchronization check is performed during transaction processing time, however, a validation is performed to make sure there is no conflicts occurred. If a conflict is found, the tentative write is discarded and the transaction is restarted.

- Divide the execution of transaction in to three phases.

Transaction execution
in the optimistic approach



read phase: data objects are read, the intended computation of the transaction is done, and writes are made on a temporary storage.

validation phase: check to see if writes made by the transaction violate the consistency of the database. If the check finds out any conflicts, the data in the temporary storage will be discarded.

Otherwise, the write phase will write the data into the database.

T: a transaction

- ts: the highest sequence number at the start of T
- tf: the highest sequence number at the beginning of its validation phase
- valid:=true;
- for t:=ts+1 to tf do
- if (writerset(t) & readset[T] != {}) then
- valid :=false;
- if valid then {write phase; increment counter, assign T a sequence number}

write phase: If the validation phase passes ok, write will be performed to the database. If the validation phase fails to pass, all temporary written data will be aborted.

